

Assignment -2

Question:

- 1.C1 extends C2
2. C1 extends C2, C3
3. C1 implements I1
4. C1 implements I1, I2
5. C1 implements C2 extends I1
6. I1 extends I2
7. I1 implements C1
8. I1 extends I2, I3

1.C1 extends C2

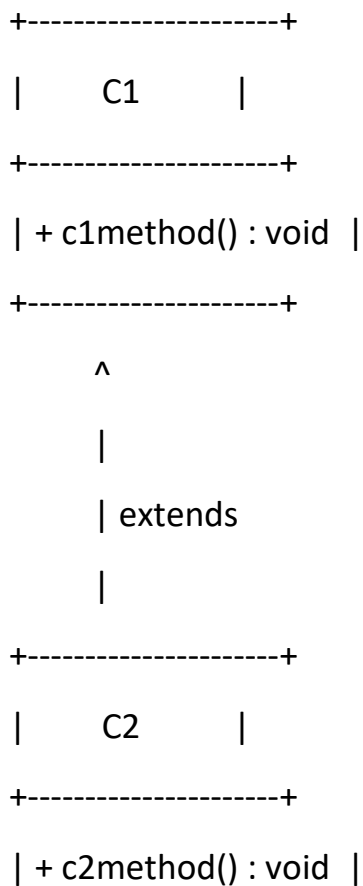
Step wise execution:

1. Start the program.
2. The program asks the user to enter a number from 1 to 8.
3. The user enters 1.
4. The switch statement selects case 1.
5. An object of C2 is created.
6. C2 extends C1, so C2 can access the method of C1.
7. c1method() is called.
8. The message from C1 is displayed.
9. c2method() is called.
10. The message from C2 is displayed.
11. The program ends.

Algorithm

1. Start.
- 2. Create class C1 with c1method().
3. Create class C2 by extending C1.
4. Define c2method() in C2.
5. Read the user's choice.
6. If the choice is 1, create an object of C2.
7. Call c1method().
8. Call c2method().
9. Display the output.
10. Stop.

UML diagram



+-----+

2. C1 extends C2, C3

Step wise execution:

1. Start the program.
2. The program asks the user to enter a number from 1 to 8.
3. The user enters 2.
4. The switch statement selects case 2.
5. The program displays the multiple inheritance example.
6. Java does not allow one class to extend two classes.
7. Therefore, C1 cannot extend both C2 and C3.
8. The program displays the explanation.
9. The program ends.

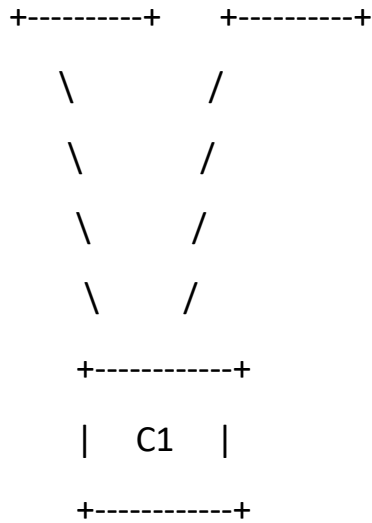
Algorithm:

1. Start.
2. Consider classes C1, C2 and C3.
3. Try to make C1 extend C2 and C3.
4. Check Java inheritance rules.
5. Java does not support multiple class inheritance.
6. Display the reason.
7. Stop.

UML diagram

+-----+ +-----+

| C2 | | C3 |



Multiple class inheritance is not supported.

3. C1 implements I1

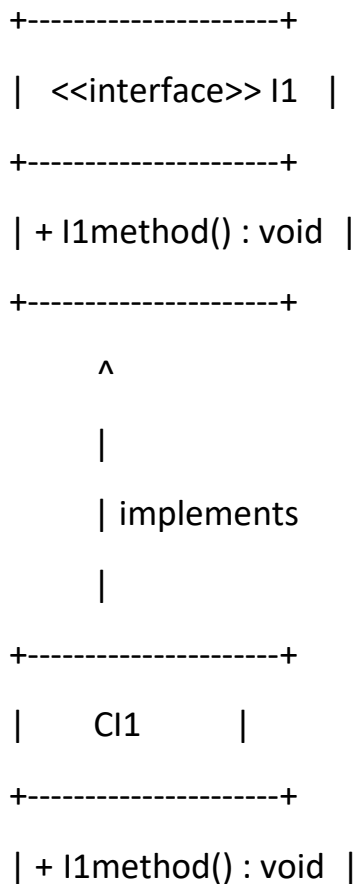
Step wise execution:

1. Start the program.
2. The program asks the user to enter a number from 1 to 8.
3. The user enters 3.
4. The switch statement selects case 3.
5. An object of C1 is created.
6. C1 implements interface I1.
7. C1 provides the implementation of I1method().
8. I1method() is called.
9. The output message is displayed.
10. The program ends.

Algorithm:

1. Start.
2. Create interface I1.
3. Declare I1method() in I1.
4. Create class CI1.
5. Implement I1 in CI1.
6. Override I1method() in CI1.
7. Create an object of CI1.
8. Call I1method().
9. Display the output.
10. Stop.

UML diagram



+-----+

4. C1 implements I1, I2

Step wise execution:

1. Start the program.
2. The program asks the user to enter a number from 1 to 8.
3. The user enters 4.
4. The switch statement selects case 4.
5. An object of CI2 is created.
6. CI2 implements both I1 and I2.
7. CI2 overrides I1method().
8. CI2 overrides I2method().
9. I1method() is called.
10. I2method() is called.
11. Both output messages are displayed.
12. The program ends.

Algorithm:

1. Start.
2. Create interface I1 with I1method().
3. Create interface I2 with I2method().
4. Create class CI2.
5. Implement I1 and I2 in CI2.
6. Override I1method().
7. Override I2method().

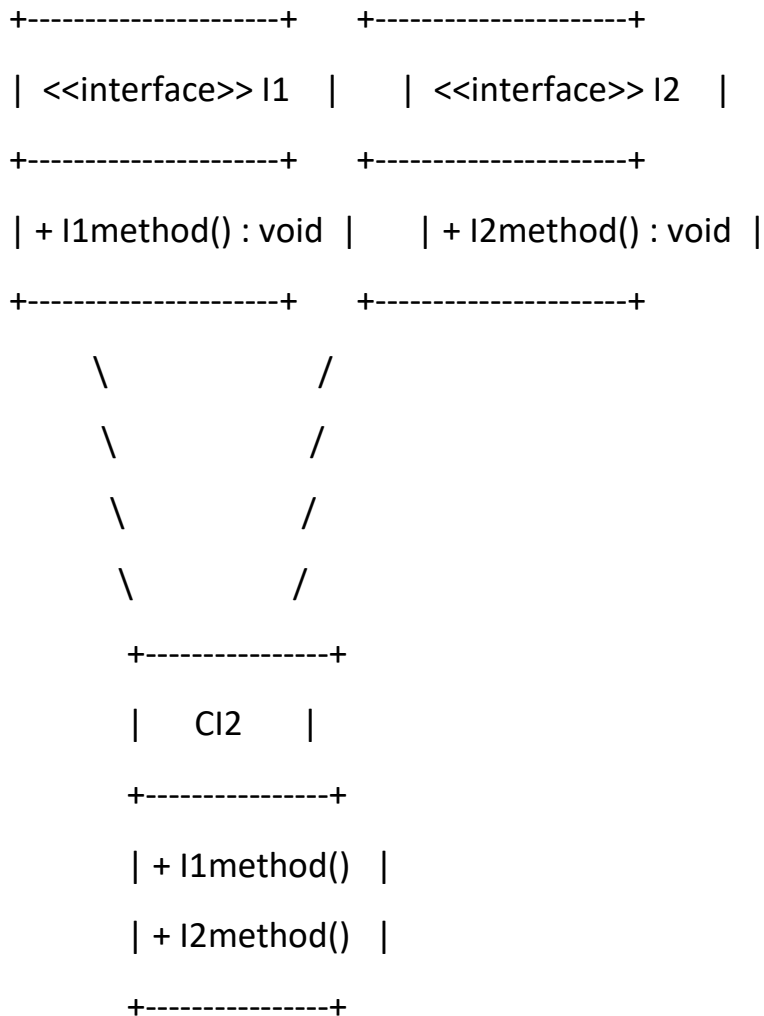
8. Create an object of CI2.

9. Call both methods.

10. Display the output.

11. Stop.

UML diagram



5. C1 implements C2 extends I1

Step wise execution:

1. Start the program.

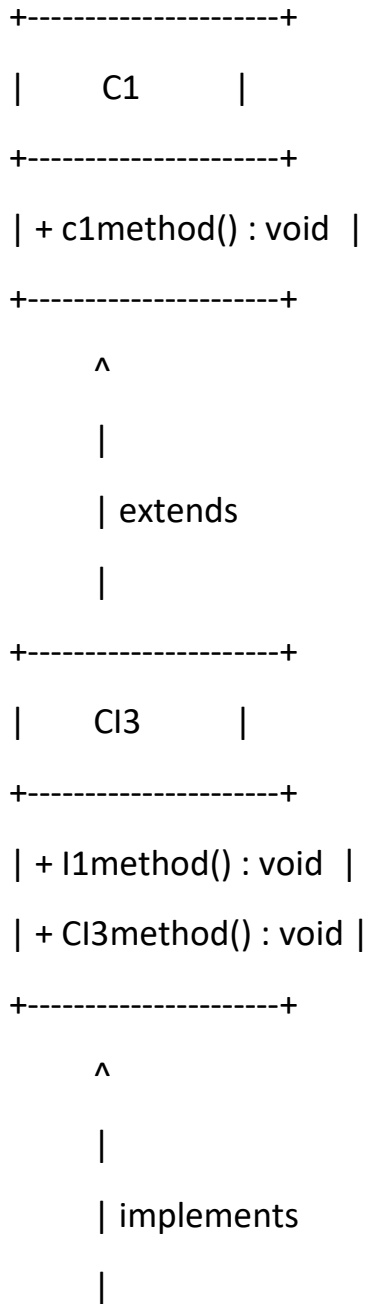
2. The program asks the user to enter a number from 1 to 8.
3. The user enters 5.
4. The switch statement selects case 5.
5. An object of C1 is created.
6. C1 extends C1.
7. C1 also implements I1.
8. C1 gets c1method() from C1.
9. C1 provides the implementation of I1method().
10. c1method() is called.
11. I1method() is called.
12. C1method() is called.
13. All output messages are displayed.
14. The program ends.

Algorithm:

1. Start.
2. Create class C1 with c1method().
3. Create interface I1 with I1method().
4. Create class C1.
5. Extend C1 using extends.
6. Implement I1 using implements.
7. Override I1method().
8. Define C1method().
9. Create an object of C1.
10. Call c1method().

11. Call I1method().
12. Call CI3method().
13. Display the output.
14. Stop.

UML diagram



```

+-----+
| <<interface>> I1 |
+-----+
| + I1method() : void |
+-----+

```

6. I1 extends I2

Step wise execution:

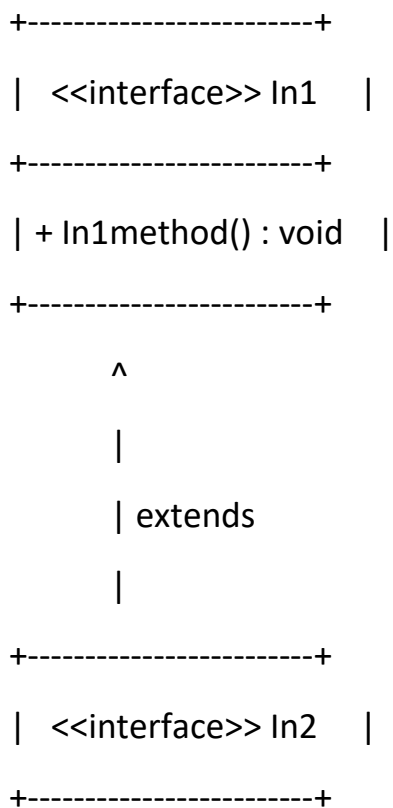
1. Start the program.
2. The program asks the user to enter a number from 1 to 8.
3. The user enters 6.
4. The switch statement selects case 6.
5. In2 extends In1.
6. CI4 implements In2.
7. CI4 must implement the methods of In1 and In2.
8. An object of CI4 is created.
9. In1method() is called.
10. In2method() is called.
11. Both output messages are displayed.
12. The program ends.

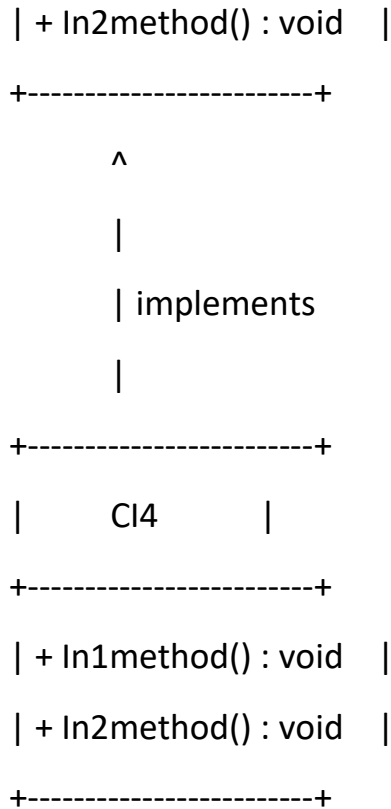
Algorithm:

1. Start.
2. Create interface In1.
3. Declare In1method() in In1.

4. Create interface In2.
5. Extend In1 using In2.
6. Declare In2method() in In2.
7. Create class CI4.
8. Implement In2 in CI4.
9. Override In1method().
10. Override In2method().
11. Create an object of CI4.
12. Call both methods.
13. Display the output.
14. Stop.

UML diagram





7. I1 implements C1

Step wise execution:

1. Start the program.
2. The program asks the user to enter a number from 1 to 8.
3. The user enters 7.
4. The switch statement selects case 7.
5. The program checks the relationship between interface I1 and class C1.
6. An interface cannot implement a class.
7. Therefore, I1 implements C1 is not allowed in Java.
8. The program displays the explanation.
9. The program ends.

Algorithm:

1. Start.
2. Consider interface I1.
3. Consider class C1.
4. Check whether an interface can implement a class.
5. Java does not allow an interface to implement a class.
6. Display the error explanation.
7. Stop.

UML diagram

```
+-----+
|      C1      |
+-----+
| + c1method() : void |
+-----+
```

X

NOT ALLOWED

```
+-----+
| <<interface>> I1 |
+-----+
| + I1method() : void |
+-----+
```

8. I1 extends I2, I3

Step wise execution:

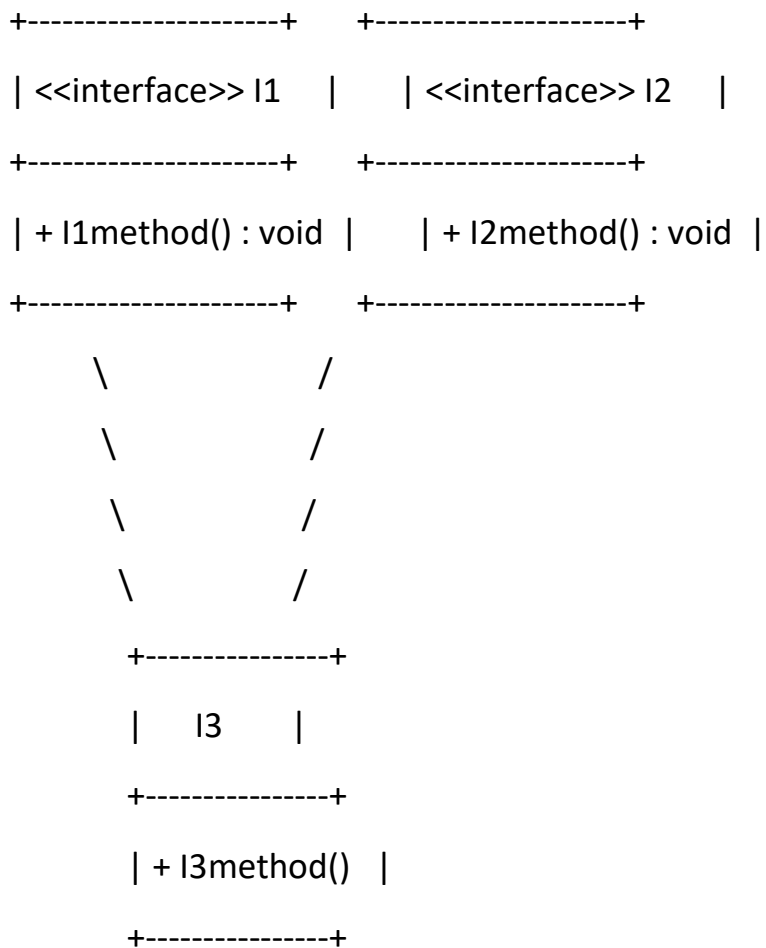
1. Start the program.
2. The program asks the user to enter a number from 1 to 8.
3. The user enters 8.
4. The switch statement selects case 8.
5. I3 extends both I1 and I2.
6. This multiple interface inheritance is allowed in Java.
7. CI321 implements I3.
8. CI321 must implement methods from I1, I2 and I3.
9. An object of CI321 is created.
10. I1method() is called.
11. I2method() is called.
12. I3method() is called.
13. All three output messages are displayed.
14. The program ends.

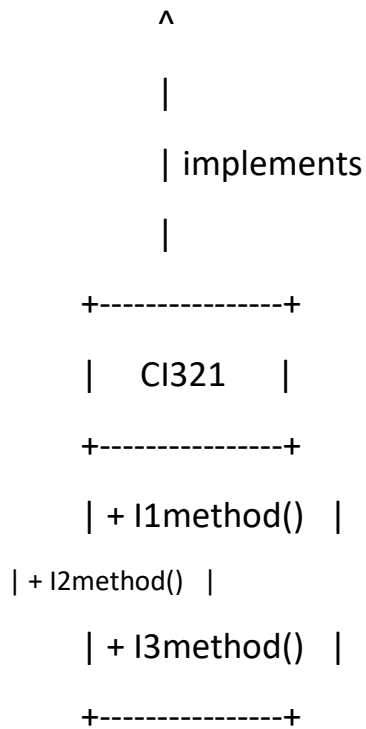
Algorithm:

1. Start.
2. Create interface I1 with I1method().
3. Create interface I2 with I2method().
4. Create interface I3.
5. Extend I1 and I2 using I3.
6. Add I3method() to I3.

7. Create class CI321.
8. Implement I3 in CI321.
9. Override I1method().
10. Override I2method().
11. Override I3method().
12. Create an object of CI321.
13. Call all three methods.
14. Display the output.
15. Stop.

UML diagram





Code:

```

package inheritance;

import java.util.Scanner;

class C1 {
    void c1method(){
        System.out.println("this is from class c1");
    }
}

//single inheritance(class to class) -1:
class C2 extends C1{
    void c2method(){
        System.out.println("this is from class c2 which is
inherited from c1");
    }
}

```



```

}

//C1 extends C2, C3 this is not possible (multiple inheritance)
-2

//interface(default abstract methods)
interface I1 {
    void I1method();
}

interface I2 {
    void I2method();
}

//. C1 implements I1 -3
class CI1 implements I1{
    @Override
    public void I1method(){
        System.out.println("this is from class CI1 which is
inherited from I1");
    }
}

// C1 implements I1, I2 -4
class CI2 implements I1,I2{
    //must be overridden for all interface methods(abstract
methods only exists in interface)

    @Override
    public void I1method(){
        System.out.println("this is from class CI2 which is
inherited from I1");
    }
    @Override
    public void I2method(){

```

```

        System.out.println("this is from class CI2 which is
inherited from I2");
    }
}

// C1 implements C2 extends I1 -5
//MULTIPLE INHERITANCE
class CI3 extends C1 implements I1{
    @Override
    public void I1method(){
        System.out.println("this is from class CI3 which is
inherited from I1");
    }

    public void CI3method(){
        System.out.println("this is from class CI3 which is
inherited from I1 and C1");
    }
}

// I1 extends I2 -6
interface In1{
    void In1method();
}

interface In2 extends In1{
    void In2method();
}

class CI4 implements In2{
    @Override
    public void In1method(){
        System.out.println("this is from class CI4 which is
inherited from In2 (in1 method)");
    }
}

```

```

        @Override
        public void In2method(){
            System.out.println("this is from class CI4 which is
inherited from In2");
        }
    }

// I1 implements C1 -7
//An interface cannot implement or extend a class.
//static interface I3 implements C1{
    //interface cannot inherit class because class is a
complete where interface is incomplete
// }

// I1 extends I2, I3 -8
//multiple inheritance
interface I3 extends I2,I1{
    void I3method();
}
//interfaces must be inherited to class to override methods
which are incomplete
class CI321 implements I3{
    @Override
    public void I1method(){
        System.out.println("this is from class CI321 which is
inherited from I1");
    }

    @Override
    public void I2method(){
        System.out.println("this is from class CI321 which is
inherited from I2");
    }
}

```

```

        @Override
        public void I3method(){
            System.out.println("this is from class CI321 which is
inherited from I3");
        }
    }
}

public class ClassInterface {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter a number from 1 to 8:");
        int choice = scanner.nextInt();

        switch (choice) {
            case 1:
                System.out.println("1. C1 extends C2");
                C2 c2 = new C2();
                c2.c1method();
                c2.c2method();
                break;
            case 2:
                System.out.println("2. C1 extends C2, C3");
                System.out.println("C1 cannot extend both C2
and C3 because Java does not support multiple class
inheritance.");
                break;
            case 3:
                System.out.println("3. C1 implements I1");
                CI1 ci1 = new CI1();
                ci1.I1method();
                break;
            case 4:
                System.out.println("4. C1 implements I1, I2");
                CI2 ci2 = new CI2();
                ci2.I1method();

```

```

        ci2.I2method();
        break;
    case 5:
        System.out.println("5. C1 implements C2 extends
I1");

        CI3 ci3 = new CI3();
        ci3.c1method();
        ci3.I1method();
        ci3.CI3method();
        break;
    case 6:
        System.out.println("6. I1 extends I2");
        CI4 ci4 = new CI4();
        ci4.In1method();
        ci4.In2method();
        break;
    case 7:
        System.out.println("7. I1 implements C1");
        System.out.println("An interface cannot extend
or implement a class such as C1.");
        break;
    case 8:
        System.out.println("8. I1 extends I2, I3");
        CI321 ci321 = new CI321();
        ci321.I1method();
        ci321.I2method();
        ci321.I3method();
        break;
    default:
        System.out.println("Invalid choice. Try
again(1-8).");
    }

    scanner.close();
}

```

}

OUTPUT Screenshot:

```
PS C:\Users\Admin\OneDrive\Documents\basicjava> 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.ClassInterface'
Enter a number from 1 to 8:
1
1. C1 extends C2
this is from class c1
this is from class c2 which is inherited from c1
PS C:\Users\Admin\OneDrive\Documents\basicjava> 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.ClassInterface'
Enter a number from 1 to 8:
2
2. C1 extends C2, C3
C1 cannot extend both C2 and C3 because Java does not support multiple class inheritance.
PS C:\Users\Admin\OneDrive\Documents\basicjava> 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.ClassInterface'
Enter a number from 1 to 8:
3
3. C1 implements I1
this is from class C11 which is inherited from I1
PS C:\Users\Admin\OneDrive\Documents\basicjava> 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.ClassInterface'
Enter a number from 1 to 8:
4
4. C1 implements I1, I2
this is from class C12 which is inherited from I1
this is from class C12 which is inherited from I2
PS C:\Users\Admin\OneDrive\Documents\basicjava> 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.ClassInterface'
Enter a number from 1 to 8:
5
5. C1 implements C2 extends I1
this is from class c1
this is from class C13 which is inherited from I1
this is from class C13 which is inherited from I1 and C1
PS C:\Users\Admin\OneDrive\Documents\basicjava> 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.ClassInterface'
Enter a number from 1 to 8:
6
```

```
5
5. C1 implements C2 extends I1
this is from class c1
this is from class C13 which is inherited from I1
this is from class C13 which is inherited from I1 and C1
PS C:\Users\Admin\OneDrive\Documents\basicjava> 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.ClassInterface'
Enter a number from 1 to 8:
6
6. I1 extends I2
this is from class C14 which is inherited from I2 (in method)
this is from class C14 which is inherited from I2
PS C:\Users\Admin\OneDrive\Documents\basicjava> 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.ClassInterface'
Enter a number from 1 to 8:
7
7. I1 implements C1
An interface cannot extend or implement a class such as C1.
PS C:\Users\Admin\OneDrive\Documents\basicjava> 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.ClassInterface'
Enter a number from 1 to 8:
8
8. I1 extends I2, I3
this is from class C1321 which is inherited from I1
this is from class C1321 which is inherited from I2
this is from class C1321 which is inherited from I3
PS C:\Users\Admin\OneDrive\Documents\basicjava> 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.ClassInterface'
Enter a number from 1 to 8:
9
Invalid choice. Try again(1-8).
PS C:\Users\Admin\OneDrive\Documents\basicjava> ^C
PS C:\Users\Admin\OneDrive\Documents\basicjava> cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.ClassInterface'
Enter a number from 1 to 8:
2
2. C1 extends C2, C3
C1 cannot extend both C2 and C3 because Java does not support multiple class inheritance.
PS C:\Users\Admin\OneDrive\Documents\basicjava>
```